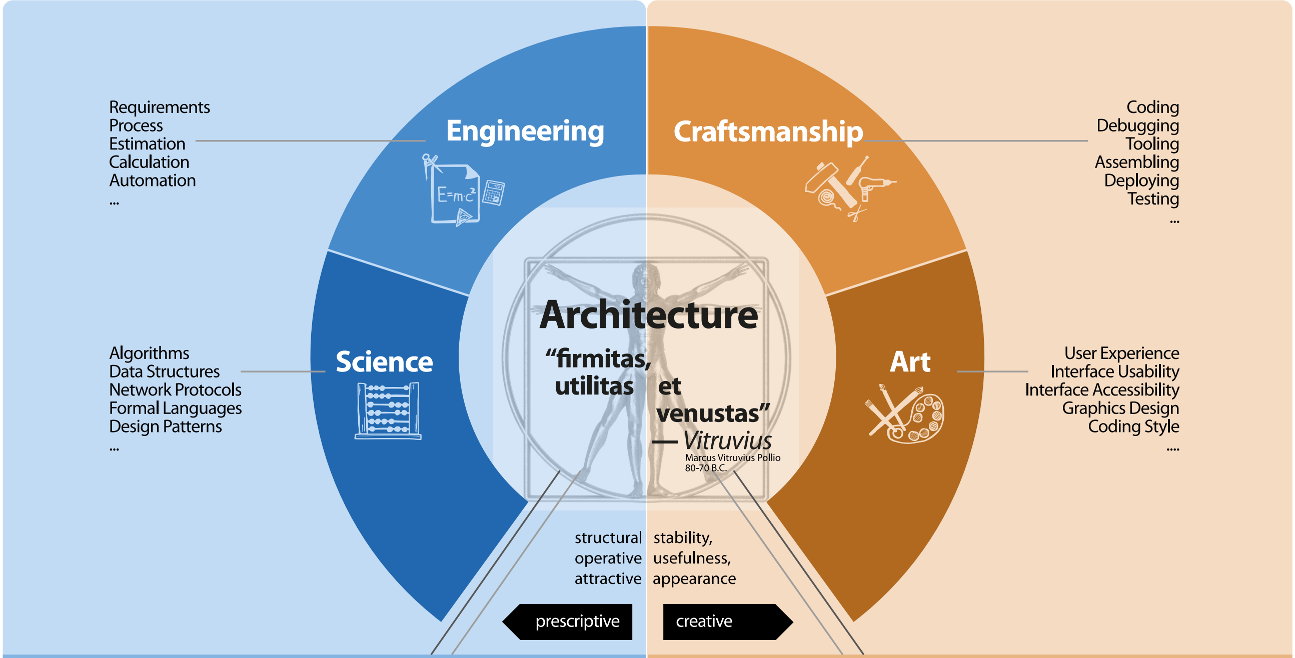
TECHNISCHE
UNIVERSITÄT
MÜNCHEN

Software Engineering in Industrial Practice (SEIP)

Dr. Ralf S. Engelschall

Architecture Stargate



Requirements
Process
Estimation
Calculation
Automation
...

Engineering

Craftsmanship

Coding
Debugging
Tooling
Assembling
Deploying
Testing
...

Algorithms
Data Structures
Network Protocols
Formal Languages
Design Patterns
...

Science

Architecture

"firmitas, utilitas et venustas"
— *Vitruvius*
Marcus Vitruvius Pollio
80-70 B.C.

Art

User Experience
Interface Usability
Interface Accessibility
Graphics Design
Coding Style
....

structural stability,
operative usefulness,
attractive appearance

prescriptive

creative

Structural Scientific Definition of Architecture:
Architecture of a system is the set of fundamental **concepts** and **properties** of the system in its environment, embodied in its **elements**[, **interfaces**], **relationships**[and **behaviours**] and the principles of its **design** and **evolution**.
(based on industry standard *ISO/IEC 42010*)

Holistic Artistic Definition of Architecture:
Great architecture is achieved **harmony** and **accord** of all **parts**, where you no longer can add, modify or remove anything without impairing the **whole**.
(based on a quote by *Leon Battista Alberti* on beauty)

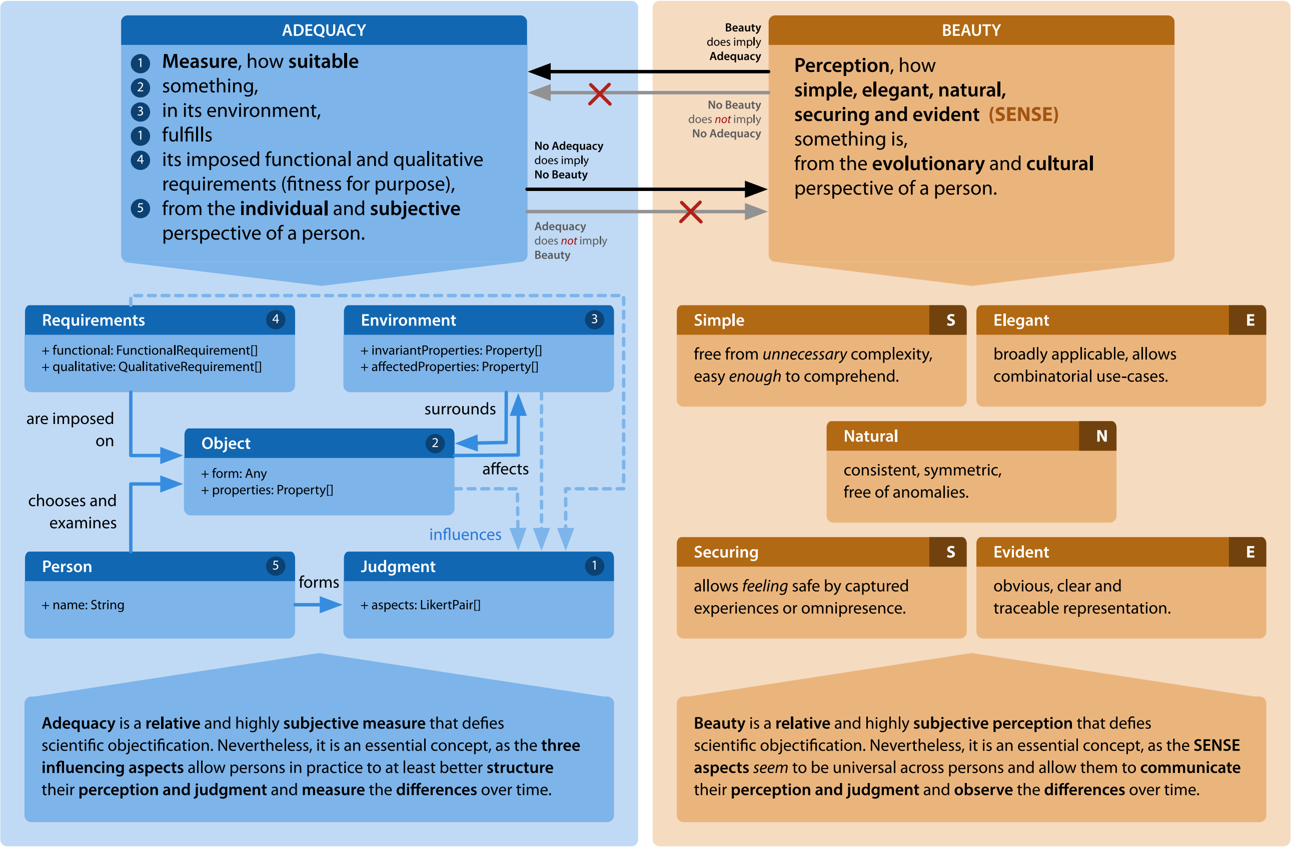
Architecture is not easy to define. You can define architecture both **structurally scientific** through measurable elements, interfaces and relationships and **wholly artistic** through "the harmony and the accord of all parts." The "truth" lies somewhere in practice in between, because the two extremes span a broad space, in which all solutions are located in practice.

On the structurally scientific side, architecture defines itself through the aspects **Science** (in particular Computer Science) and **Engineering** (especially Software Engineering). On the holistic artistic side, architecture defines itself through the aspects of **Craftsmanship** (especially programming) and **Art** (especially User Experience).

Questions

- ? How to define **Architecture**?
- ? What are the four **Aspects** of **Architecture**?

Adequacy and Beauty



ADEQUACY

① **Measure**, how **suitable**
② something,
③ in its environment,
① fulfills
④ its imposed functional and qualitative requirements (fitness for purpose),
⑤ from the **individual** and **subjective** perspective of a person.

Beauty does imply Adequacy

No Beauty does *not* imply No Adequacy

No Adequacy does imply No Beauty

Adequacy does *not* imply **Beauty**

BEAUTY

Perception, how
simple, elegant, natural, securing and evident (SENSE)
something is,
from the **evolutionary** and **cultural** perspective of a person.

Requirements ④
+ functional: FunctionalRequirement[]
+ qualitative: QualitativeRequirement[]

Environment ③
+ invariantProperties: Property[]
+ affectedProperties: Property[]

Object ②
+ form: Any
+ properties: Property[]

Person ⑤
+ name: String

Judgment ①
+ aspects: LikertPair[]

Simple S
free from *unnecessary* complexity, easy *enough* to comprehend.

Elegant E
broadly applicable, allows combinatorial use-cases.

Natural N
consistent, symmetric, free of anomalies.

Securing S
allows *feeling* safe by captured experiences or omnipresence.

Evident E
obvious, clear and traceable representation.

Adequacy is a **relative** and highly **subjective measure** that defies scientific objectification. Nevertheless, it is an essential concept, as the **three influencing aspects** allow persons in practice to at least better **structure** their **perception and judgment** and **measure** the **differences** over time.

Beauty is a **relative** and highly **subjective perception** that defies scientific objectification. Nevertheless, it is an essential concept, as the **SENSE aspects** *seem* to be universal across persons and allow them to **communicate** their **perception and judgment** and **observe** the **differences** over time.

Intellectual Content: Version 1.0.2 (2022-10-09), Authored 2021-2022 by Dr. Ralf S. Engelschall, based on pre-work of Stefan Brütsch
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Adequacy is defined as the measure, how suitably something, in its environment, fulfills its imposed functional and qualitative requirements (fitness for purpose), from the individual and subjective perspective of a person.

Adequacy is a relative and highly subjective measure that defies scientific objectification. Nevertheless, adequacy is an essential concept, as the three influencing aspects (Requirements, Environment, Object) allow persons in practice to at least better structure their perception and judgment and measure the differences over time.

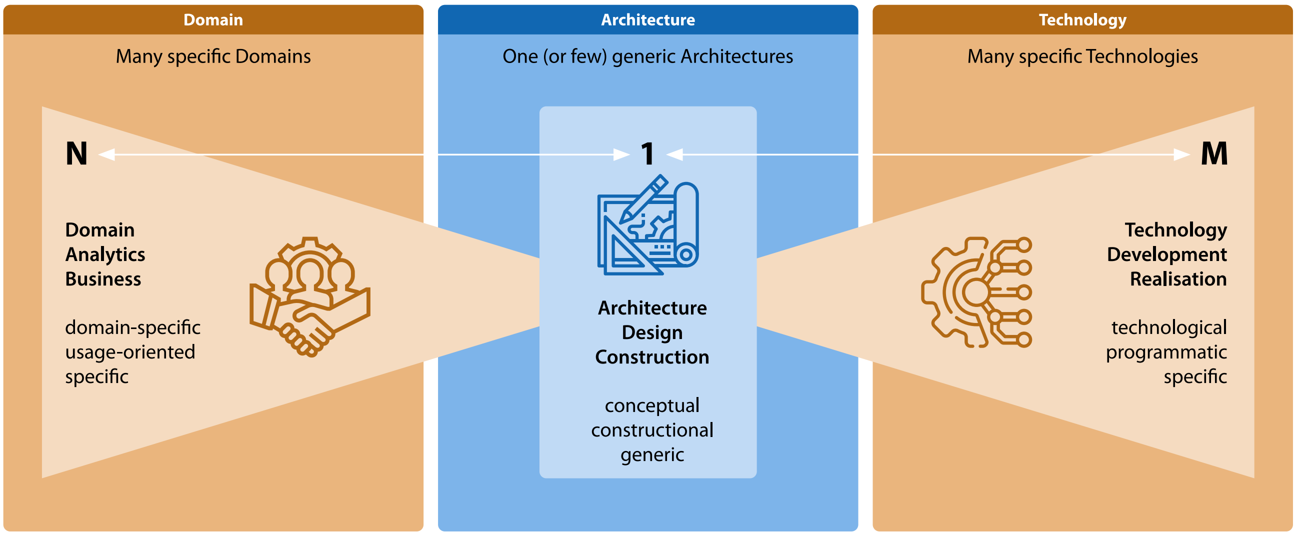
Beauty is defined as the perception, how simple, elegant, natural, securing and evident (SENSE) something is, from the evolutionary and cultural perspective of a person.

Beauty is a relative and highly subjective perception that defies scientific objectification. Nevertheless, beauty is an essential concept, as the SENSE aspects seem to be universal across persons and allow them to communicate their perception and judgment and observe the differences over time.

Questions

- ? Is it possible to measure Adequacy or Beauty in general?
- ? Is it possible to measure Adequacy or Beauty in the context of a single person?

King Discipline Architecture



Domain

Many specific Domains

N

Domain
Analytics
Business

domain-specific
usage-oriented
specific

Architecture

One (or few) generic Architectures

1

Architecture
Design
Construction

conceptual
constructional
generic

Technology

Many specific Technologies

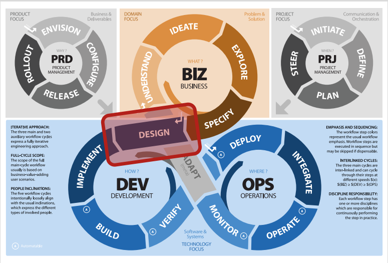
M

Technology
Development
Realisation

technological
programmatic
specific

Software Engineering Disciplines

Software Engineering Workflow



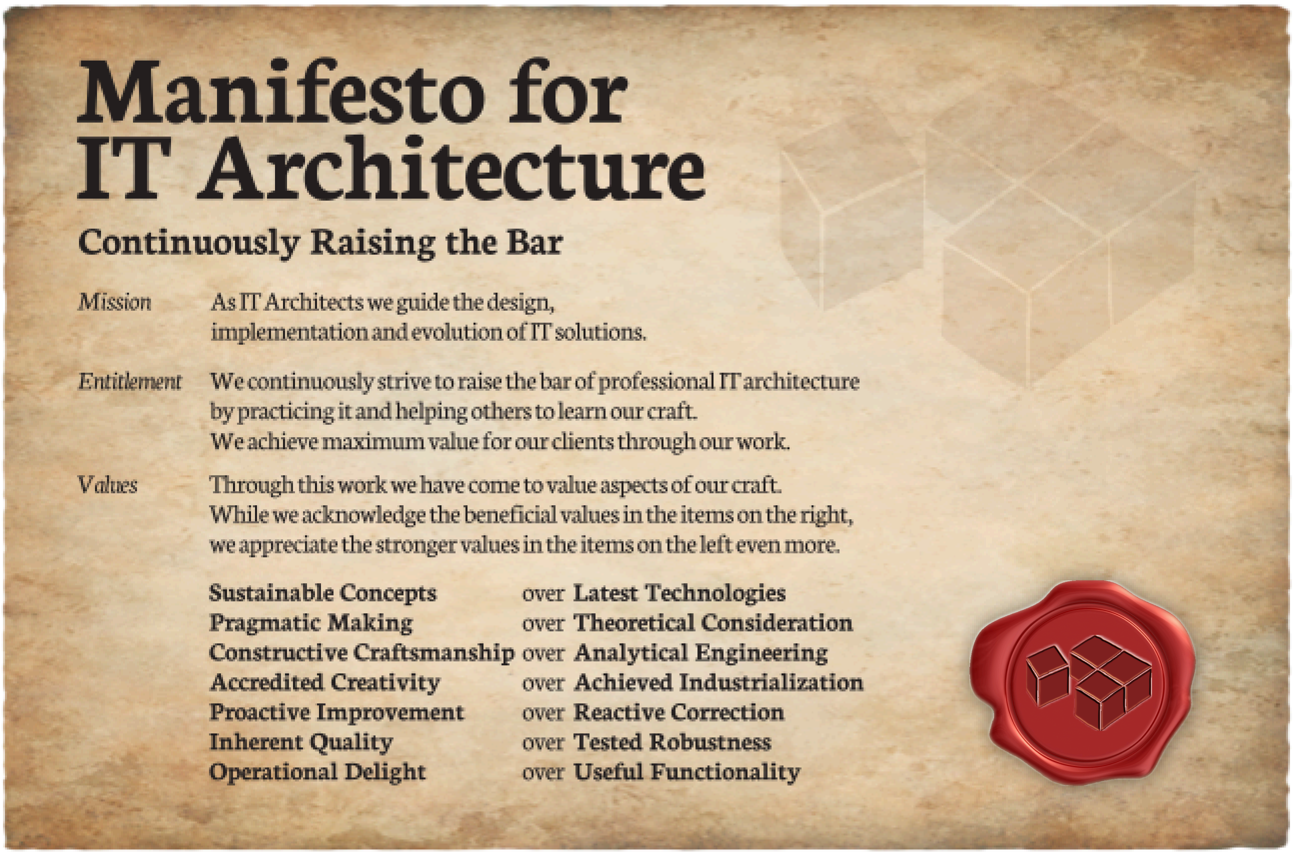
(Software) **Architecture** is considered the "King Discipline" in **Software Engineering**, since it is the central, general, and conceptual link between the many, potential, specific, realized **Domains** and the many, potential, specific, realizing **Technologies**. The architectural construction of an application takes place in the logical step "Design" within the BizDevOps workflow of Software Engineering.

Questions

? Why is **Architecture** considered the "King Discipline" of Software Engineering?

Intellectual Content: Version 1.0.0 (2022-10-09), Authored 2022 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.0 (2022-10-09), Copyright © 2022 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.

Architecture Manifesto



Manifesto for IT Architecture

Continuously Raising the Bar

Mission As IT Architects we guide the design, implementation and evolution of IT solutions.

Entitlement We continuously strive to raise the bar of professional IT architecture by practicing it and helping others to learn our craft.
We achieve maximum value for our clients through our work.

Values Through this work we have come to value aspects of our craft.
While we acknowledge the beneficial values in the items on the right,
we appreciate the stronger values in the items on the left even more.

Sustainable Concepts over **Latest Technologies**
Pragmatic Making over **Theoretical Consideration**
Constructive Craftsmanship over **Analytical Engineering**
Accredited Creativity over **Achieved Industrialization**
Proactive Improvement over **Reactive Correction**
Inherent Quality over **Tested Robustness**
Operational Delight over **Useful Functionality**

The **Manifesto for IT Architecture** is a policy statement for IT architecture. First and foremost, it calls for "Continuously Raising the Bar", since after just 50 years of Software Engineering and Software Architecture, even though we already know a number of best practices, the discipline will certainly have to continue to develop for a very long time.

The Mission for IT architects is the design, implementation, and evolution of IT solutions. The Entitlement is to continuously raise the bar and help others to learn the "craft." Naturally, through the work of architects, the maximum added value is achieved for customers.

The basic values, which play a central role in this craft and which are greatly appreciated, are: **Latest Technologies**, **Theoretical Consideration**, **Analytical Engineering**, **Achieved Industrialization**, **Reactive Correction**, **Tested Robustness** and **Useful Functionality**.

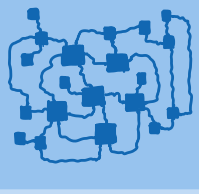
In addition, there are additional values, which also play a central role and are even more appreciated: **Sustainable Concepts** (the content of **Architecture Fundamentals**!), **Pragmatic Making**, **Constructive Craftsmanship**, **Accredited Creativity**, **Proactive Improvement**, **Inherent Quality** and **Operational Delight**.

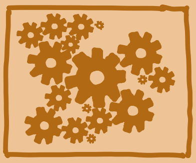
Questions

- ? (none)

Complex vs. Complicated

	complex	complicated
FOCUS	refers to the **extrinsic** and **higher**- or **macro**-level difficulty of a system,	refers to the **intrinsic** and **lower**- or **micro**-level difficulty of a system,
RATIONALE	because the system involves many different and **connected** parts	because the system involves many different and **difficult** aspects
CHALLENGE	which take time to **comprehend** and **master** in **total**,	which take time to **understand** and **learn** in **detail**,
INSIGHT	and which nevertheless are **easy** to explain.	and which usually are **hard** to explain.





NOTICE	NOTICE
Simple (non-complicated) systems can be **complex**.	**Clear** (non-complex) systems can be **complicated**.

RECOGNIZE	RECOGNIZE
Architecture primarily has to master the **complex** aspects of a system.	**Development** primarily has to master the **complicated** aspects of a system.

“Complex” refers to the extrinsic and higher- or macro-level difficulty of a system because the system involves many different and connected parts which take time to comprehend and master in total, and which nevertheless are easy to explain.

“Complicated” refers to the intrinsic and lower- or micro-level difficulty of a system because the system involves many different and difficult aspects which take time to understand and learn in detail, and which usually are hard to explain.

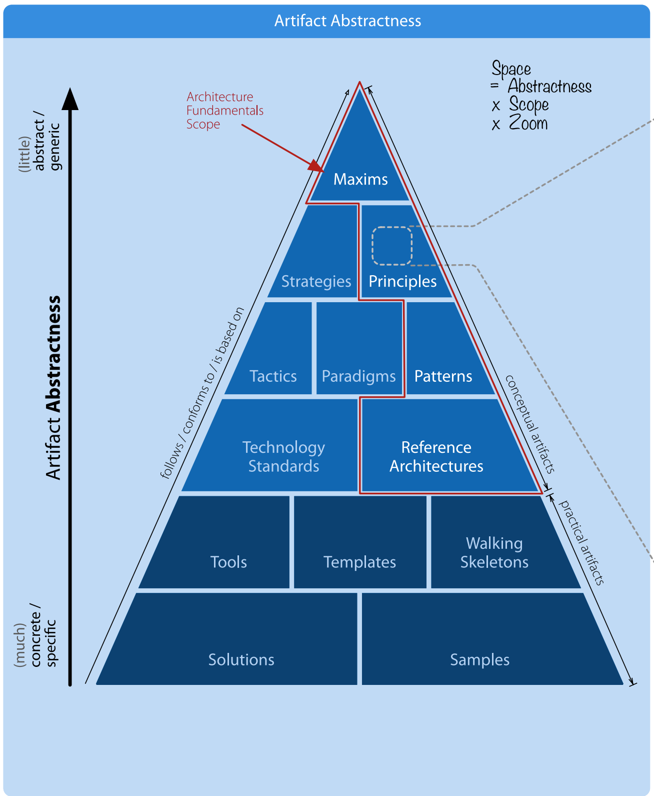
Note: Simple (non-complicated) systems can be complex – clear (non-complex) systems can be complicated.

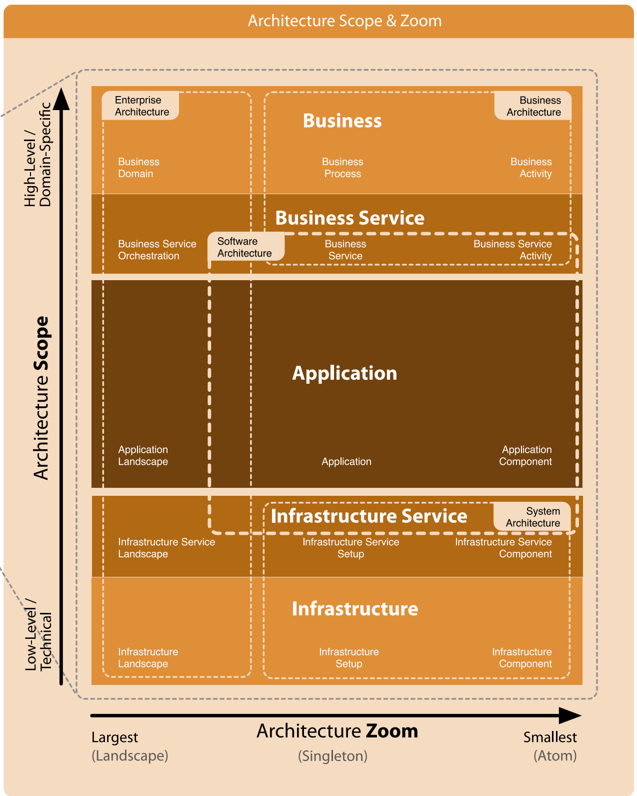
The crucial difference is: The architecture or the construction has to master the complex aspects of a system. The development or realization has to master the complicated aspects of a system.

Questions

- ? Does **Architecture** primarily have to deal with **complex** or **complicated** aspects of a system?

Architecture Space





The **IT Architecture Space** consists of three dimensions: the **Artifact Abstractness** (a degree of abstraction of all artifacts that are known by the architect), the **Architecture Scope** (the field of architecture in which one acts) and the **Architecture Zoom** (the detail level of architecture in which one acts).

One primarily distinguishes between three Architecture Scopes: (high-level/domain-specific) **Business**, **Application** and (low-level/technical) **Infrastructure**. In addition, the two secondary Architecture Scopes **Business Service** and **Infrastructure Service** are used to reduce in practice the "mental leap" from Business to Application and from Application to Infrastructure.

On each Architecture Scope, one can choose from at least three different **Architecture Zooms**: from **Landscape** (largest), over **Singleton** to **Atom** (smallest).

In the space of Architecture Scope and Zoom, one differentiates between four types of **IT architecture**: **Business Architecture**, **Software Architecture**, **System Architecture** and **Enterprise Architecture**.

The higher/lower the level of abstraction of the artifacts, the larger/smaller usually is the coverage of the room made from Architecture Scope and Architecture Zoom.

Questions

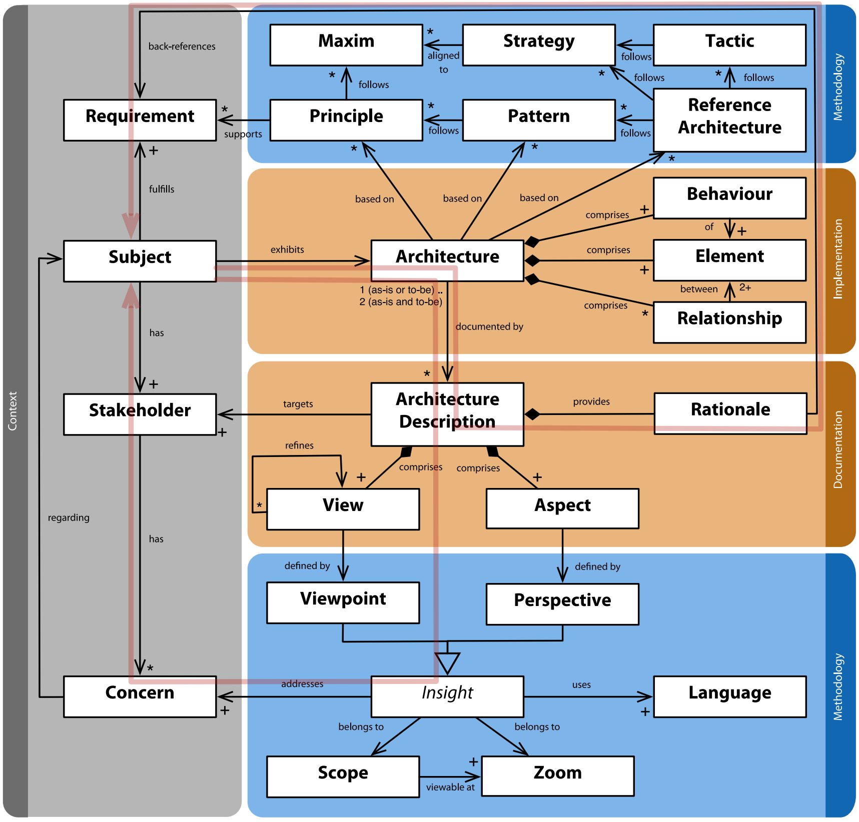
- ? The IT **Architecture Space** consists of which three dimensions?
- ? Which four types of Architecture are known in IT?

Intellectual Content: Version 1.0.9 (2020-06-23), Authored 2011-2020 by Dr. Ralf S. Engelschall and Marc Mährländer, based on pre-work of msg Applied Technology Research
Graphical Illustration: Version 1.1.2 (2019-09-15), Copyright © 2011-2019 Dr. Ralf S. Engelschall <http://engelschall.com>. All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

ARCHITECTURE
FUNDAMENTALS

Architecture Ontology

TECHNISCHE
UNIVERSITÄT
MÜNCHEN



Requirement: A functional or non-functional demand or imposed obligation on the **Subject**.

Subject: Any type of business process, business service, software application, infrastructure service or infrastructure setup.

Stakeholder: Person, group or entity with an interest in or **Concerns** about the **Architecture**.

Concern: Requirement, objective, intention, or aspiration a **Stakeholder** has on an **Architecture**.

Architecture Description: Set of artifacts that document an **Architecture** in a way **Stakeholders** can understand and ensures their **Concerns** are met.

Viewpoint: Collection of templates and conventions for constructing one type of **View**. It addresses **Concerns** and contains guidelines for constructing a View.

Perspective: Collection of decisions, guidelines and rules that ensure a **Subject** exhibits a set of non-functional **Requirements**, considered across a number of **Views**.

Insight: Superordinate abstract concept to address particular **Concerns** through **Viewpoints** and **Perspectives**.

Architecture: Inherent static and dynamic structure of a **Subject** which comprise **Elements**, the visible **Behaviour** of **Elements** and **Relationships** between **Elements**.

Element: Fundamental part from which a **Subject** can be considered to be constructed.

Behaviour: Run-time characteristic of **Architecture Elements** of the **Subject**.

Relationship: Static or dynamic relationship between **Elements** of an **Architecture** of the **Subject**.

Rationale: Fundamental reasons for a particular chosen **Architecture**, usually strongly based on non-functional **Requirements**.

View: Representation of one or more structural aspects of an **Architecture** that illustrates how the **Architecture** addresses one or more **Concerns**.

Aspect: Representation of one or more non-functional aspects of an **Architecture** that illustrate how the **Architecture** addresses one or more **Concerns**.

Language: Formal, semi-formal or even prose language to describe an Insight, so it optimally addresses the **Concerns** of **Stakeholders**.

Maxim: Fundamental, generally valid set of values and rules to guide the architecture discipline (think: law).

Strategy: Long-term situation-independent plan of approaches to achieve a particular goal (think: war). Aligned and not in conflict with any **Maxims**.

Tactic: Short-term situation-dependent plan of actions to achieve a particular goal (think: battle). Following and supporting a **Strategy**.

Principle: Fundamental truth, rule, tenet or policy an **Architecture** follows.

Pattern: Proven recurring theme, structure, approach or behavior an **Architecture** and its **Elements** can follow.

Reference Architecture: Reusable proven Architecture template, **based** on a set of Patterns and following one or more **Tactics** and **Strategies**.

Scope: Primary area of the **Architecture** space an **Insight** addresses: either Business, Business Service, Application, Infrastructure Service or Infrastructure.

Zoom: Level of view distance to the **Architecture** an **Insight** has. Also known as the **Insight** detail level, ranging from smallest atoms, over singletons to the largest landscape.

Intellectual Content: Version 1.1.8 (2020-08-13), Authored 2010-2020 by Dr. Ralf S. Engelschall, inspired by ISO/IEC/IEEE 42010:2011, IEEE 1471:2000 and The Open Group TOGAF 9
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

So that Architects can communicate meaningfully in practice, one must agree on a few basic terms and their meaning. The terms are defined in a taxonomy and are described in the **Architecture Ontology** in relation to each other.

In the Architecture Ontology, there are two main important "loops." Both start at the **Subject**, which has an **Architecture**, which is documented via the **Architecture Description**.

Loop 1: The Architecture Description gives **Rationales** for decisions, which, ideally, should be back-referencing to **Requirements**. Because an Architecture Description should not document the **WHAT** but the **WHY**. Because the WHAT can also be seen in the code, but the WHY not!

Loop 2: The Architecture Description consists of **Views** and **Aspects**, which methodically are called **Viewpoints** and **Perspectives**. Both together provide **Insights** at **Scope** and **Zoom Level** (see Architecture Space!) and are documented via a specific (graphical or textual) **Language**. In any case, only those insights will be given which address a **Concern** of a **Stakeholder**. One also doesn't program anything which one doesn't need!

Questions

- ? What should an **Architecture Description** document besides the **WHAT**?
- ? What should an **Architecture Description** especially address through **Insights**?

Requirements Basics

Intellectual Content: Version 1.0.2 (2022-10-28), Authored 2018-2022 by Dr. Ralf S. Engelschall, based on ideas from BABOK 2.0:2009, IEEE Std 610:1990 and IEEE Std 830–1998
Graphical Illustration: Version 1.0.2 (2022-10-09), Copyright © 2018-2022 Dr. Ralf S. Engelschall <http://engelschall.com>. All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Requirements Specification

A binding document that specifies the requirements for a solution, by focusing on the WHAT and WHY of the solution — and *not* giving instructions for the HOW.

The documented set of requirements has to be: correct, unambiguous, complete, consistent, ranked, verifiable, modifiable, and traceable.

Requirement Classes

FR **Functional** (Shall Do)

A condition or capability that a solution must have to provide its service in terms of its behaviour and information. Think: Functionality.

NFR **Non-Functional** (Shall Be)

A condition, property or quality that a solution must have to satisfy a contract, standard, or other formally imposed obligation. Think: Constraints and "*-ilities".

Requirement Interdependencies

POS **Positive** (Backing)

One requirement supports the other (e.g. for NFRs: Maintainability and Comprehensibility usually support Adaptability, Portability, Modifiability, etc., and Scalability usually supports Availability, etc.)

NEG **Negative** (Trade-Off)

One requirement interferes with the other (e.g. for NFRs: Security usually interferes with Efficiency, Usability, Performance, etc., and Orthogonality can interfere with Usability)

Requirement Characteristics

S **Specific**

The requirement is precise, unambiguous, and clear on what should be done.

M **Measurable**

The requirement can be verified when it has been achieved by use of a particular test.

A **Achievable**

The requirement is achievable given existing circumstances and feasible and viable solutions.

R **Relevant**

The requirement is relevant to the goals of the context.

T **Time-Bound**

The requirement can be achieved within a reasonable time frame.

Requirement Life-Time

E **Enduring**

The requirement lasts forever, as it is derived from core activities and organisational structures.

V **Volatile**

The requirement can be temporary, as it might change over time.

Requirement Expression

[<req-id>] <req-name>**:**
<subject/actor>
SHALL
<result/action/condition>
BECAUSE
<rationale>

The **Requirements Specification** is a binding document in which primarily the WHAT and WHY of the solution are specified, however not the concrete technical HOW. The set of requirements must be correct, unambiguous, complete, consistent, prioritized, verifiable, changeable and traceable.

There are two types of requirements: **Functional Requirement** ("Shall Do", functionality) and **Non-Functional Requirements** ("Shall Be", conditions, in English often expressed with words ending in "-ility"). The architect primarily takes care of the latter.

Requirements can also be reciprocally positive (backing) or negative (trade-off). The architect also primarily takes care of the latter.

Requirements should be "SMART": **Specific**, **Measurable**, **Achievable**, **Relevant** and **Time-Bound**.

In addition, requirements are either **Enduring** (fixed) or **Volatile** (unstable). The architect should pay attention to the latter.

Questions

? What kind of **requirements** should the architect primarily keep in mind and should be explicitly addressed by him in the solution finding?

ARCHITECTURE
FUNDAMENTALS

Non-Functional Requirements

TECHNISCHE
UNIVERSITÄT
MÜNCHEN

Compliance

- **CMP Compliance** Ability to meet rules and standards
- **CRT Certification** Ability to confirm certain characteristics
- **LCN Licensing** Ability to permit to own and use something
- **PRC Pricing** Ability to have reasonable price and permit charging for a product

Operation

- **OPR Operability** Ability to be reasonably operated
- **SPP Supportability** Ability to be reasonably supported
- **MNT Maintainability** Ability to cope with changing environments and requirements
- **TST Testability** Ability to be completely and repeatably tested
- **TRC Traceability** Ability to track the path something takes
- **MSR Measurability** Ability to measure characteristics according to defined metrics

Usability

- **USB Usability** Ability for ease of use, user-friendliness, accessibility, convenience, intuitiveness
- **CPY Comprehensability** Ability to be easily understood
- **ACC Accessibility** Ability to be used by people with disabilities.

• • •

Correctness

- **PRD Predictability** Ability to predict state and behaviour under run-time
- **FDL Fidelity** Ability to reproduce state and behaviour of the real world
- **RLV Relevance** Ability to serve as a means to a given purpose
- **PRN Precision** Ability to be exact and accurate in operation
- **CRS Correctness** Ability to be algorithmically correct with respect to the specification
- **PRV Provability** Ability to mathematically prove algorithmical correctness

Protection

- **SFT Safety** Ability to protect against undeliberate failures, errors and accidents
- **SEC Security** Ability to protect against deliberate destruction, damage and harm

Availability

- **AVL Availability** Ability to be operationally available anytime
- **UBQ Ubiquity** Ability to be operationally present anywhere
- **RPT Repeatability** Ability to repeat state and behaviour in sequence
- **RPR Reproducability** Ability to reproduce state and behaviour from scratch
- **RCV Recoverability** Ability to recover state and behaviour after a disastrous failure

• • •

Quality

- **RLB Reliability** Ability to perform required functions under stated conditions for a specified time
- **RSL Resilience** Ability to provide an acceptable level of service in face of faults and challenges
- **RBS Robustness** Ability to withstand stress, pressure, or changes in procedure or circumstances
- **STB Stability** Ability to not suffer from internal failures in service
- **DRB Durability** Ability to keep interfaces and functionality as is for a period of time
- **INT Integrity** Ability to keep state consistency and avoid data corruption

Performance

- **PRF Performance** Ability to efficiently perform work, i.e., with a good work to time & resource ratio
- **SCL Scalability** Ability to scale mostly linearly with changing requirements or conditions
- **RSP Responsiveness** Ability to respond quickly to external interaction

Structure

- **SMP Simplicity** Ability to be plain, natural, straight-forward and with no observable complexity
- **FLX Flexibility** Ability to be easily modifyable in order to respond to altered circumstances
- **MDL Modularity** Ability to consist of individually comprehensible modules
- **ORT Orthogonality** Ability to follow great separation of concerns in design

• • •

Execution

- **EFF Efficiency** Ability to perform work in the most economical way: good input/output ratio
- **EFC Efficacy** Ability to perform work in order to getting things done and meeting targets
- **EFV Effectiveness** Ability to perform the "right" work by setting right targets to achieve goals

Interfacing

- **ITY Interoperability** Ability to correctly operate and exchange information with foreign components
- **CPT Compatibility** Ability to correctly operate despite expected older or newer interfaces

Evolution

- **RSB Reusability** Ability to reuse code or data with slight or no modifications
- **ADP Adaptability** Ability to cope with smaller changes in the run-time environment
- **PRT Portability** Ability to cope with larger changes in run-time environment
- **CFG Configurability** Ability to individualize state and behaviour by non-destructive instructions
- **CST Customizability** Ability to individualize state and behaviour by possibly destructive instructions
- **EXT Extensibility** Ability to extend state and behaviour in a controlled way
- **TLR Tailorability** Ability to adjust state and behaviour in a controlled way
- **MDF Modifiability** Ability to change state and behaviour in an arbitrary way

• • •

Intellectual Content: Version 1.0.9 (2020-08-13), Authored 2009-2020 by Dr. Ralf S. Engelschall, based on ISO/IEC FDIS 9126-1:2000
Graphical Illustration: Version 1.1.1 (2019-07-18),

There are potentially many **Non-Functional Requirements.** For any solution, one must therefore first determine the actual quantity of such Requirements. This quantity must be minimized by the Architect!

For every contractually stipulated Requirement, one should take into account that it is clearly defined since there are great similarities between Requirements and the differences are sometimes very subtle.

A few of the Non-Functional Requirements that almost always have to be considered in practice are **Maintainability**, **Usability**, **Security**, **Availability**, **Reliability**, **Performance**, **Responsiveness** and **Adaptability**.

Questions

❓ Name 3 in practice frequently considered **Non-Functional Requirements**!